

OOP

UNIT 6 – OPERATOR OVERLOADING

Operator overloading allows user-defined classes to behave like built-in types. Instead of calling specially named member functions, we can often write clearer and more natural code using C++ operators.

LEARNING OBJECTIVES

- Explain why operator overloading exists.
- Distinguish between member and non-member operators.
- Implement operators correctly.
- Understand when overloaded operators improve readability.
- Dynamic memory management with Rule Of Three.

WHY DO WE NEED OPERATOR OVERLOADING?

In previous units we manipulated Song objects through member functions. As programs become larger, repeatedly calling comparison functions becomes less readable.

Without operator overloading:

```
if(first.sameAs(second))
{
    cout << "Same song" << endl;
}
```

With operator overloading:

```
if(first == second)
{
    cout << "Same song" << endl;
}
```

The second version is easier to read because it resembles comparisons between built-in types such as int or string.

WHAT IS OPERATOR OVERLOADING?

An overloaded operator is simply a function with a special name. The compiler translates the operator syntax into a function call.

```
first == second;
```

// equivalent to

```
first.operator==(second);
```

OPERATOR==

Our first overloaded operator determines whether two Song objects represent the same song.

```
class Song {
private:
    string title;
    string artist;
    int duration;
public:
    Song(string title,string artist,int duration);
    bool operator==(const Song& other) const;
};
bool Song::operator==(const Song& other) const {
    return title == other.title &&
           artist == other.artist &&
           duration == other.duration;
}
int main() {
    Song first("Imagine","John Lennon",183);
    Song second("Imagine","John Lennon",183);

    if(first == second)
        cout << "Equal" << endl;
    else
        cout << "Different" << endl;
}
```

Output

Equal

The parameter is passed by const reference because the comparison should not modify the second object and copying is unnecessary. The member function itself is also const because comparing two songs does not change either object.

REUSE EXISTING OPERATORS

Suppose the Song class is later extended with an album name. If both `operator==` and `operator!=` contain independent comparison logic, both functions must be updated. Forgetting one of them introduces inconsistent behavior.

```
bool Song::operator==(const Song& other) const {  
    return title == other.title &&  
           artist == other.artist &&  
           duration == other.duration &&  
           album == other.album;  
}  
  
bool Song::operator!=(const Song& other) const {  
    return !(*this == other);  
}
```

Only one function contains the comparison rules. The second function automatically follows every future change.

RELATIONAL OPERATORS

Equality tells us whether two objects represent the same value. Sometimes we also need to order objects. For example, a music application may sort songs alphabetically before displaying them.

```
Song first("Halo", "Beyonce", 261);  
Song second("Imagine", "John Lennon", 183);
```

```
if(first < second)  
    cout << "Halo appears first alphabetically." << endl;
```

Let's implement operator< and use it as the foundation for the remaining relational operators.

OPERATOR<

To sort Song objects, the compiler must know what it means for one song to be smaller than another. In this example, songs are ordered alphabetically by their title.

```
bool Song::operator<(const Song& other) const {  
    return title < other.title;  
}  
  
int main() {  
    Song first("Halo", "Beyonce", 261);  
    Song second("Imagine", "John Lennon", 183);  
  
    if(first < second)  
        cout << "Halo comes first." << endl;  
}
```

Output

```
Halo comes first.
```

The comparison relies on the string class. Since string already supports the relational operators, we can compare the titles directly.

OPERATOR>

Once operator< has been implemented, there is no reason to duplicate the comparison logic.

```
bool Song::operator>(const Song& other) const {  
    return other < *this;  
}
```

The operands are reversed. If the other object is smaller than the current object, then the current object must be greater.

OPERATOR<=

```
bool Song::operator<=(const Song& other) const {  
    return !(other < *this);  
}
```

The result is true whenever the current object is either smaller than or equal to the second object.

OPERATOR>=

CHOOSING ORDERING

The *meaning* of operator< should be intuitive and **consistent**.

Different applications may require different ordering rules.

A music library might sort by title, while another program might sort by artist or duration.

Regardless of the chosen criterion, the comparison should produce a predictable ordering.

```
bool Song::operator<(const Song& other) const
{
    if(title != other.title)
        return title < other.title;

    return artist < other.artist;
}
```

This implementation compares titles first. If two titles are identical, the artist name is used to break the tie. Such multi-level comparisons are common in real applications.

CONSISTENCY FOR RELATIONAL OPERATORS

All comparison operators should describe the same logical ordering. For example, if two Song objects are considered equal by operator==, neither object should be considered smaller than the other.

```
Song first("Imagine", "John Lennon", 183);  
Song second("Imagine", "John Lennon", 183);  
  
cout << (first == second) << " , ";  
cout << (first < second) << " , ";  
cout << (first > second) << endl;
```

Output

```
1, 0 , 0
```

These results are **consistent**. The two objects are equal, so neither one precedes the other.

By expressing the remaining relational operators in terms of operator<, every ordering rule is maintained in one place. If the ordering criteria change later, only operator< needs to be modified.

MEMBER OR NON-MEMBER OPERATORS?

So far every overloaded operator has been implemented as a **member** function. Many operators naturally belong to the left operand, making a member function an appropriate choice. However, C++ also allows operators to be implemented as ordinary non-member functions .

The following comparison operators are commonly implemented as member functions because the current object is the left operand.

```
bool Song::operator==(const Song& other) const;  
bool Song::operator!=(const Song& other) const;  
bool Song::operator<(const Song& other) const;  
bool Song::operator>(const Song& other) const;
```

When the compiler evaluates `first < second`, the expression is interpreted as:

```
first.operator<(second);
```

The left operand becomes the current object, while the right operand is passed as the function parameter.

NON-MEMBER OPERATOR FUNCTIONS

Not every operator should be a member function. Sometimes neither operand should receive special treatment. In those cases, an ordinary function may provide a cleaner design.

```
class Fraction {
private:
    int numerator;
    int denominator;
public:
    Fraction(int n = 0, int d = 1);

    int getNum() const;
    int getDenom() const;
};

bool operator==(const Fraction& left,    const Fraction& right)
{
    return left.getNum() * right.getDenom() ==
           right.getNum() * left.getDenom();
}

Fraction a(1,2);
Fraction b(2,4);

if(a == b)
    cout << "Equal fractions" << endl;
```

The compiler translates this expression into a call to the non-member function.

WHEN IS OPERATOR MEMBER/NON-MEMBER?

Member Function	Non-member Function
The left operand is naturally the current object.	Both operands should be treated equally.
Needs direct access to private members.	Can rely on the public interface.
Frequently used for assignment, subscript, and function call operators.	Frequently used for arithmetic and comparison operators.
Has direct access to the class's private/protected members.	Accesses class data through public methods (unless declared <i>friend</i>).
The left operand must be an object of the class.	Allows implicit conversions on both operands.
Examples: =, [], (), ->, +=, -=	Examples: +, -, *, /, ==, <, >

Rule of thumb:

- Use **member functions** for operators that modify or are tightly coupled to the object, and for operators that must be members (=, [], (), ->).
- Use **non-member functions** for *symmetric* operators where both operands should be treated equally (+, -, ==, <, etc.).

Some non-member operators will later require access to **private** data. The next page introduces **friend** functions, which provide controlled access when it is genuinely needed.

FRIEND OPERATOR FUNCTIONS

A non-member operator can use only the public interface of a class. Sometimes this is exactly what we want. In other situations, the operator needs direct access to private data members. Instead of making those members public, C++ allows selected functions to become friends of the class.

```
class Fraction
{
private:
    int numerator;
    int denominator;

public:
    Fraction(int n = 0, int d = 1);

    friend bool operator==(const Fraction& left,
                           const Fraction& right);
};
```

The friend declaration does not create a member function. It simply **grants** the specified non-member function **permission** to access the private members of the class.

```
bool operator==(const Fraction& left, const Fraction& right)
{
    return left.numerator * right.denominator ==
           right.numerator * left.denominator;
}
```

The function is still called exactly like any other overloaded operator.

```
Fraction a(1,2);
Fraction b(2,4);

cout << (a == b) << endl;
```

Output

```
1
```

The compiler translates the expression into a call to the friend function, while allowing it to access the private data members directly.

WHEN SHOULD FRIEND BE USED?

Friendship should be used only when it simplifies the design without weakening encapsulation unnecessarily. Many operators can be implemented through the public interface and do not require friendship. Friendship is granted explicitly by the class. It should be viewed as a design decision rather than a shortcut around access control.

Friend is Appropriate	Friend is Usually Unnecessary
The operator needs direct access to private members.	Public getters provide all required information.
Performance or simplicity would suffer through the public interface.	The operator can be implemented naturally using existing public member functions.
Stream operators often become friends.	Comparison operators frequently do not.
Used when external functions need privileged access to class internals.	Preferable when encapsulation can be maintained.
Can make implementation more efficient and straightforward.	Keeps class interfaces cleaner and reduces coupling.

Rule of thumb:

- Use **friend** when an external operator or function genuinely requires access to private data and using the public interface would be awkward or inefficient.
- Avoid **friend** when the same functionality can be achieved cleanly through the class's public interface, preserving encapsulation.

STREAM OPERATORS

Objects frequently need to be displayed on the screen or read from the keyboard. If we rely only on member functions such as `print()`, every class will require a different interface. The stream insertion and extraction operators make user-defined classes behave like built-in types.

After overloading these operators, objects can be used naturally with `cout` and `cin`.

```
Song song("Imagine", "John Lennon", 183);  
  
cout << song << endl;  
cin >> song;
```

The expressions above resemble the syntax used for integers, doubles and strings. This consistency is one of the major advantages of operator overloading.

OPERATOR<<

The insertion operator places an object's information into an output stream. Because the left operand is an ostream object rather than a Song object, this **operator cannot be implemented** as a member of Song.

```
class Song {
private:
    string title;
    string artist;
    int duration;

public:
    Song(string t , string a , int d )/*init list*/{ };

    friend ostream& operator<<(ostream& out, const Song& song);
};

ostream& operator<<(ostream& out, const Song& song) {
    out << song.title << " - " << song.artist
        << " (" << song.duration << " sec)";

    return out;
}
```

The function returns the output stream by **reference**. Returning the stream allows multiple insertion operations to be chained together.

```
Song first("Halo", "Beyonce", 261);  
Song second("Imagine", "John Lennon", 183);  
  
cout << first << endl  
      << second << endl;
```

Output

```
Halo - Beyonce (261 sec)  
Imagine - John Lennon (183 sec)
```

Notice that the operator itself does not print endl. The caller decides whether to continue printing on the same line or move to the next line.

WHY RETURN OSTREAM&?

The expression `cout << first << second` consists of two consecutive operator calls. The first call returns the same output stream, which immediately becomes the left operand of the second call.

```
((cout << first) << second);
```

If `operator<<` returned `void`, the second insertion would be impossible because there would be no stream object to continue the expression.

OPERATOR>>

Reading objects from the keyboard should be just as natural as displaying them. The extraction operator allows the programmer to use `cin` with user-defined classes in the same way as built-in types.

Since the left operand is an `istream` object, `operator>>` is implemented as a non-member function. It is commonly declared as a friend so that it can update the private data members of the object.

```
class Song {
private:
    string title;
    string artist;
    int duration;

public:
    Song(string t , string a , int d )/*init list*/{ };

    friend istream& operator>>(istream& in, Song& song);
};

istream& operator>>(istream& in, Song& song) {
    in >> song.title;
    in >> song.artist;
    in >> song.duration;

    return in;
}
```

Notice that the Song object is not const. The extraction operator **modifies** the object by storing new values in its data members.

```
Song song;  
  
cin >> song;  
  
cout << song << endl;
```

This implementation assumes that the title and artist contain a single word.

CHAINING INPUT OPERATIONS

Just like `operator<<`, the extraction operator returns the stream by reference. This allows several input operations to appear in a single statement.

```
Song first;  
Song second;  
  
cin >> first >> second;
```

The compiler interprets the expression as two consecutive calls. The first call returns `cin`, which immediately becomes the left operand of the second call.

```
((cin >> first) >> second);
```

COMMON MISTAKES WITH STREAM OPERATORS

Mistake	Correct approach
Return <code>ostream</code> or <code>istream</code> by value.	Return them by reference.
Declare <code>operator>></code> with <code>const Song&</code> .	Use <code>Song&</code> because the object is modified.
Implement <code>operator<<</code> as a <code>Song</code> member.	Implement it as a non-member function.
Print <code>endl</code> inside <code>operator<<</code> .	Let the caller decide the formatting.

With stream operators implemented, `Song` objects integrate naturally with the standard input and output streams. The next operators extend this idea even further by allowing objects to participate in arithmetic expressions.

ARITHMETIC OPERATORS

The next group of operators allows objects to participate in **calculations** and expressions. When overloaded carefully, arithmetic operators make code easier to read because they express the programmer's intent directly.

OPERATOR+

Suppose we want to combine two playlists. A natural interpretation is that adding two Playlist objects creates a new playlist containing the songs of both operands. Neither original object should be modified, so the operator returns a new object.

```
class Playlist
{
private:
    vector<Song> songs;

public:
    void addSong(const Song& song)
    {
        songs.push_back(song);
    }
}
```



```

Playlist operator+(const Playlist& other) const
{
    //Both operands remain unchanged
    Playlist result = *this;
    // Creates and returns a new object
    for(const Song& song : other.songs)
        result.addSong (song);
    //Allows temporary results in larger expressions
    return result;
}
};

```

The implementation begins by copying the current playlist. The songs from the second playlist are then appended to the copy. Finally, the new object is returned, leaving both operands unchanged.

```

Playlist favorites;
Playlist classics;

Playlist allSongs = favorites + classics;

```

The expression resembles integer addition even though a completely different operation is taking place.

WHY RETURN A NEW OBJECT?

Users generally expect the + operator to produce a result without changing its operands. This behavior matches the semantics of built-in numeric types and makes expressions predictable.

OPERATOR+=

Unlike operator+, the compound assignment operator modifies the current object. It performs the same logical operation but stores the result directly in the left operand.

```
Playlist& Playlist::operator+=(const Playlist& other)
{
    for(const Song& song : other.songs)
        songs.push_back(song);

    return *this;
}
```

Returning *this by reference allows assignment operators to be chained just like the built-in types.

```
favorites += classics;
favorites += recent;
```

A common implementation strategy is to implement operator+= first and then define operator+ in terms of it. This avoids duplicating the logic for combining two objects.

OPERATOR+ USING OPERATOR+=

A common design technique is to implement the compound assignment operator first and then reuse it inside operator+. This avoids maintaining two separate implementations of the same operation.

```
Playlist& Playlist::operator+=(const Playlist& other) {  
    for(const Song& song : other.songs)  
        songs.push_back(song);  
    return *this;  
}  
  
Playlist Playlist::operator+(const Playlist& other) const {  
    Playlist result = *this;  
    result += other;  
    return result;  
}
```

The actual logic for appending songs exists in only one place. If the implementation changes in the future, only operator+= needs to be updated.

OPERATOR-

Operators should represent **meaningful** actions. For a Playlist class, subtraction can be interpreted as removing every song that appears in another playlist.

```
Playlist Playlist::operator-(const Playlist& other) const {  
  
    Playlist result;  
  
    for(const Song& song : songs) {  
        bool found = false;  
  
        for(const Song& otherSong : other.songs) {  
            if(song == otherSong)  
            {  
                found = true;  
                break;  
            }  
        }  
  
        if(!found)  
            result.addSong(song);  
    }  
  
    return result;  
}
```

The operator creates a new playlist. Every song from the current playlist is examined. Songs that also appear in the second playlist are skipped, while the remaining songs are copied into the result.

```
Playlist favorites;  
Playlist downloaded;  
  
Playlist remaining = favorites - downloaded;
```

OPERATOR-=

As with addition, the compound assignment version modifies the current object instead of creating a new one.

```
Playlist& Playlist::operator-=(const Playlist& other)  
{  
    *this = *this - other;  
    return *this;  
}
```

This implementation reuses operator-. Although not always the most efficient approach, it keeps the code short and avoids duplicating the removal logic.

DESIGNING MEANINGFUL OPERATORS

Not every operator is appropriate for every class. Before overloading an operator, ask whether its meaning would be immediately understood by someone reading the code.

Good design	Poor design
<code>playlist1 + playlist2</code> combines playlists.	<code>playlist1 * playlist2</code> combines playlists.
<code>price1 < price2</code> compares values.	<code>price1 % price2</code> compares prices.
<code>vector += item</code> appends logically.	<code>vector ^ item</code> performs an unrelated action.

A well-chosen operator improves readability because the code resembles natural language. A poorly chosen operator forces readers to remember artificial meanings and makes programs harder to maintain.

DYNAMYC MEMORY MANAGEMENT AND RULE OF THREE

Suppose that Playlist stores its songs in a dynamically allocated array. Because the class now owns dynamic memory, it must correctly copy and destroy that resource.

```
class Playlist
{
private:
    Song* songs;
    int size;
    int capacity;

    static const int INIT_CAPACITY = 10;

public:
    Playlist(int capacity = INIT_CAPACITY);
    Playlist(const Playlist& other);
    Playlist& operator=(const Playlist& other);
    ~Playlist();
};
```

CONSTRUCTOR

```
Playlist::Playlist(int capacity)
    : songs(new Song[capacity]), size(0), capacity(capacity) {}
```

DESTRUCTOR

```
Playlist::~~Playlist()
{
    delete[] songs; // delete resource owned by the class
}
```

COPY CONSTRUCTOR

```
Playlist::Playlist(const Playlist& other)
    : songs(new Song[other.capacity]),
      size(other.size),
      capacity(other.capacity)
{
    for (int i = 0; i < size; i++) //copy data
        songs[i] = other.songs[i];
}
```

ASSIGNMENT OPERATOR

```
Playlist& Playlist::operator=(const Playlist& other)
{
    if(this == &other) return *this; //avoid self copy

    delete[] songs;    //remove old resource

    capacity = other.capacity;    // copy other objects' fields
    size = other.size;
    songs = new Song[capacity];    // create new resource

    for(int i=0;i<size;i++)        // copy data
        songs[i]=other.songs[i];

    return *this;
}
```

RULE OF THREE

The Rule of Three says: *if your class needs a custom destructor, copy constructor, or copy assignment operator, it probably needs all three*, because the class manages a resource that requires proper copying and cleanup.

Also there are **Rule of Five + Rule of Zero**

THE SUBSCRIPT OPERATOR (OPERATOR[])

Some classes represent collections of objects. In these situations, it is convenient to access individual elements using the same square **bracket** notation that is used with arrays and vectors.

Suppose a `Playlist` stores its songs in a vector. Instead of creating a member function such as `getSong()`, we can overload `operator[]` and make the syntax shorter and more intuitive.

```
class Playlist {
private:
    vector<Song> songs;

public:
    Song& operator[](size_t index)    {
        return songs[index];
    }

    const Song& operator[](size_t index) const    {
        return songs[index];
    }
};
```

Two versions of the operator are provided. The non-const version is selected when the playlist can be modified, while the const version is used for read-only access.

```
Playlist playlist;

playlist[0].setTitle("Imagine"); // non-const version

cout << playlist[0] << endl; // const version
```

Because the operator returns a reference, the selected Song object can be modified directly. Returning by value would modify only a temporary copy.

WHY RETURN A REFERENCE?

```
Song& Playlist::operator[](size_t index) {
    return songs[index];
}
```

A reference refers to the actual object stored inside the playlist. As a result, assignments affect the original element.

```
playlist[0] = Song("Halo", "Beyonce", 261);
```

If operator[] returned a Song instead of Song&, the assignment above would replace only a temporary object, leaving the playlist unchanged.

ADDING BOUNDS CHECKING

The implementation shown so far assumes that the supplied index is valid. In real applications, invalid indices should be detected.

```
Song& Playlist::operator[](size_t index) {  
    if(index >= songs.size())  
        throw out_of_range("Invalid index");  
  
    return songs[index];  
}
```

Throwing an exception prevents access outside the valid range of the collection. Exception handling will be studied in a later unit, but this example illustrates why many standard containers perform bounds checking through dedicated member functions.

WHEN SHOULD OPERATOR[] BE OVERLOADED?

Appropriate	Usually inappropriate
Classes representing indexed collections.	Classes representing a single value.
Arrays, vectors, matrices, playlists.	Dates, fractions, bank accounts.
Objects where an integer index identifies an element.	Objects with no natural indexing operation.

`Operator[]` should represent **random access** to an element. When no such concept exists, another member function usually communicates the programmer's intent more clearly.

THE FUNCTION CALL OPERATOR (OPERATOR())

The function call operator allows an object to be used as if it were a function. Unlike most operators, `operator()` may accept any number of parameters, making it extremely flexible.

Although it is widely used in the Standard Library, it can also simplify the design of user-defined classes.

```
class Playlist {
private:
    vector<Song> songs;

public:
    int operator()(const string& artist) const {
        int count = 0;
        for(const Song& song : songs)
            if(song.getArtist() == artist)
                count++;
        return count;
    }
};
```

In this implementation, calling a Playlist object **counts how many** songs belong to a particular artist.

```
Playlist playlist;
cout << playlist("Queen") << endl;
cout << playlist("ABBA") << endl;
```

The syntax resembles an ordinary function call even though the operation is performed by an object.

MULTIPLE PARAMETERS

The function call operator is not limited to a single argument. It can receive several parameters and interpret them in any meaningful way.

```
int Playlist::operator()(const string& artist, int minDuration) const
{
    int count = 0;
    for(const Song& song : songs)    {
        if(song.getArtist() == artist && song.getDuration() >= minDuration) {
            count++;
        }
    }
    return count;
}

cout << playlist("Queen", 200) << endl; // Counts songs by the specified
artist whose duration is at least the given number of seconds.
```

WHEN SHOULD OPERATOR() BE USED?

Appropriate uses	Poor uses
Objects that naturally perform an operation.	Ordinary getters and setters.
Objects that compute a result from parameters.	Operations whose meaning is unclear.

Operator() should improve readability rather than hide the purpose of the code. If a normal member function communicates the operation more clearly, that design is usually preferable.

UNARY OPERATORS

Unary operators work with a **single** operand. They are useful when a class represents a value that can be negated, tested, incremented, or decremented. To keep the examples consistent, this section uses the Fraction class introduced earlier in the unit.

```
class Fraction {
private:
    int numerator;
    int denominator;

public:
    Fraction(int n = 0, int d = 1)
        : numerator(n), denominator(d) {}

    Fraction operator+() const;
    Fraction operator-() const;
    bool operator!() const;

    Fraction& operator++();
    Fraction operator++(int);
    Fraction& operator--();
    Fraction operator--(int);

    friend ostream& operator<<(ostream& out,
                               const Fraction& value);
};
```

The unary arithmetic and logical operators do not require a second Fraction parameter. The increment and decrement operators modify the current object, so their return types follow the conventions used by built-in numeric types.

UNARY PLUS (OPERATOR+)

Unary plus normally represents the unchanged value of an object. It does not modify the fraction. Although this operator is rarely necessary, it demonstrates how a unary operator is declared and implemented.

```
Fraction Fraction::operator+() const {  
    return *this;  
}  
Fraction value(3, 4);  
Fraction copy = +value;  
  
cout << copy << endl;
```

Output

```
3/4
```

The function is const because it does not change the current object. It returns a Fraction by value so that the result behaves like an independent numeric value.

UNARY MINUS (OPERATOR-)

Unary minus creates the additive inverse of a fraction. Negating 3/4 produces -3/4. The original object should remain unchanged.

```
Fraction Fraction::operator-() const {  
    return Fraction(-numerator, denominator);  
}  
Fraction value(3, 4);  
  
cout << -value << endl;  
cout << value << endl;
```

Output

```
-3/4  
3/4
```

The first statement displays the temporary Fraction returned by operator-. The second statement confirms that the original object was not modified.

LOGICAL NOT (OPERATOR!)

The logical NOT operator should represent a clear true-or-false condition. For Fraction, a natural interpretation is to ask whether the fraction represents zero.

```
bool Fraction::operator!() const {  
    return numerator == 0;  
}  
Fraction first(0, 5);  
Fraction second(3, 4);  
  
if(!first)  
    cout << "First is zero." << endl;  
  
if(!second)  
    cout << "Second is zero." << endl;
```

Output

```
First is zero.
```

Only the first condition is true. A fraction is zero whenever its numerator is zero, provided that its denominator is valid.

PREFIX INCREMENT (OPERATOR++)

Incrementing a fraction adds one to its value. For a fraction n/d , adding one produces $(n + d)/d$. Prefix increment modifies the object first and then returns the modified object.

```
Fraction& Fraction::operator++() {  
    numerator += denominator;  
    return *this;  
}  
Fraction value(3, 4);  
  
cout << ++value << endl;  
cout << value << endl;
```

Output

```
7/4  
7/4
```

Prefix increment returns `Fraction&` because the result is the current object after it has been modified. Returning `*this` by reference also allows expressions such as `++(++value)`.

POSTFIX INCREMENT (OPERATOR++(INT))

Postfix increment must return the value that existed before the object was modified. The unused int parameter distinguishes the postfix version from the prefix version.

```
Fraction Fraction::operator++(int) {  
    Fraction oldValue = *this;  
    ++(*this);  
    return oldValue;  
}  
Fraction value(3, 4);  
  
cout << value++ << endl;  
cout << value << endl;
```

Output

```
3/4  
7/4
```

The function first saves a copy of the original object. It then reuses prefix increment to perform the actual modification. Finally, it returns the saved value by value.

PREFIX AND POSTFIX INCREMENT COMPARED

Expression	Value Produced	Object Afterward
<code>++value</code>	The incremented value	Incremented
<code>value++</code>	The original value	Incremented

Both forms modify the object. The difference is the value produced by the expression. Prefix increment is generally more efficient because it does not need to preserve and return the old value.

PREFIX DECREMENT (OPERATOR--)

Decrementing a fraction subtracts one from its value. For n/d , subtracting one produces $(n - d)/d$.

```
Fraction& Fraction::operator--() {  
    numerator -= denominator;  
    return *this;  
}  
Fraction value(7, 4);  
  
cout << --value << endl;  
cout << value << endl;
```

Output

```
3/4
3/4
```

POSTFIX DECREMENT (OPERATOR--(INT))

Postfix decrement returns the original value while still modifying the object. As with postfix increment, the dummy int parameter exists only to distinguish the postfix form.

```
Fraction Fraction::operator--(int) {
    Fraction oldValue = *this;
    --(*this);
    return oldValue;
}
Fraction value(7, 4);

cout << value-- << endl;
cout << value << endl;
```

Output

```
7/4
3/4
```

The postfix implementation reuses prefix decrement. Keeping the modification logic in one function reduces duplication and prevents the two versions from becoming inconsistent.

COMPLETE UNARY OPERATOR EXAMPLE

The following example combines the unary operators in one complete program. The stream operator is included so that Fraction objects can be displayed directly.

```
#include <iostream>
using namespace std;

class Fraction {
private:
    int numerator;
    int denominator;

public:
    Fraction(int n = 0, int d = 1)
        : numerator(n), denominator(d) {}

    Fraction operator+() const {
        return *this;
    }

    Fraction operator-() const {
        return Fraction(-numerator, denominator);
    }

    bool operator!() const {
        return numerator == 0;
    }
}
```

```

}

Fraction& operator++() {
    numerator += denominator;
    return *this;
}

Fraction operator++(int) {
    Fraction oldValue = *this;
    ++(*this);
    return oldValue;
}

Fraction& operator--() {
    numerator -= denominator;
    return *this;
}

Fraction operator--(int) {
    Fraction oldValue = *this;
    --(*this);
    return oldValue;
}

friend ostream& operator<<(ostream& out,
                           const Fraction& value) {
    out << value.numerator << "/"

```



```

        << value.denominator;
    return out;
}
};

int main() {
    Fraction value(3, 4);
    Fraction zero(0, 5);

    cout << +value << endl;
    cout << -value << endl;

    if(!zero)
        cout << "Zero fraction" << endl;

    cout << ++value << endl;
    cout << value++ << endl;
    cout << value << endl;

    cout << --value << endl;
    cout << value-- << endl;
    cout << value << endl;

    return 0;
}

```

Output

```
3/4
-3/4
Zero fraction
7/4
7/4
11/4
7/4
7/4
3/4
```

RETURN TYPES FOR UNARY OPERATORS

Operator	Typical Return Type	Reason
+value	Fraction	Produces an independent unchanged value.
-value	Fraction	Produces a new negated value.
!value	bool	Answers a true-or-false question.
++value	Fraction&	Returns the modified current object.
value++	Fraction	Returns a saved copy of the old value.
--value	Fraction&	Returns the modified current object.
value--	Fraction	Returns a saved copy of the old value.

COMMON MISTAKES WITH UNARY OPERATORS

Mistake	Correct Approach
Modify the object inside unary + or unary -.	Return a new value and keep the original unchanged.
Return prefix increment by value.	Return Fraction& and return *this.
Return postfix increment by reference.	Return the saved old value by value.
Duplicate the modification logic in postfix.	Reuse the prefix implementation.
Forget the dummy int postfix parameter.	Use operator++(int) or operator--(int).
Give operator! an unclear meaning.	Use it only for a natural logical condition.

WHEN SHOULD UNARY OPERATORS BE OVERLOADED?

Unary operators are appropriate when their meaning follows naturally from the class. Fractions behave like numeric values, so negation, increment, and decrement are intuitive. A class such as Song does not have an obvious meaning for unary minus or increment, so overloading those operators would make the program harder to understand.

Appropriate	Usually Inappropriate
-fraction negates a numeric value.	-song has no obvious meaning.
++fraction adds one.	++playlist may surprise the reader.
!fraction tests whether it is zero.	!song is unclear without special documentation.

The same principle applies to every overloaded operator: the syntax should communicate the operation clearly without requiring the reader to memorize an artificial definition.

CONVERSION OPERATORS

Sometimes an object should be converted naturally into another type. C++ allows classes to overload conversion operators so that such conversions can occur explicitly or, in some situations, implicitly. Conversion operators should be used carefully because automatic conversions can make code harder to understand if they are overused.

```
class Song {
private:
    int duration;

public:
    Song(string t = "", string a = "", int d = 0);

    explicit operator int() const {
        return duration;
    }
};
```

The operator has no return type because the target type appears as part of the operator name. In this example, converting a Song to an int returns its duration.

```
Song song("Imagine", "John Lennon", 183);
int seconds = static_cast<int>(song);
cout << seconds << endl;
```

Output

183

WHY USE EXPLICIT?

Without the `explicit` keyword, the compiler may perform conversions automatically whenever an integer is expected. Although this can sometimes be convenient, it can also introduce subtle programming errors.

```
void printD(int seconds);  
  
Song song("Halo", "Beyonce", 261);  
  
// Explicit conversion  
printD(static_cast<int>(song));
```

Using `explicit` makes the programmer's intention clear and prevents unexpected implicit conversions.

OPERATORS THAT CANNOT BE OVERLOADED

Most C++ operators may be overloaded, but a small number are built into the language itself and cannot be redefined.

Operator	Reason
::	Scope resolution is part of the language syntax.
.	Direct member access cannot be changed.
.*	Pointer-to-member access is built into the language.
?:	The conditional operator has fixed semantics.
sizeof	Determined by the compiler.
Typeid	Provides runtime type information.

GENERAL GUIDELINES FOR OPERATOR OVERLOADING

Operator overloading should improve readability rather than surprise the reader. Overload an operator only when its meaning closely matches the behavior programmers already expect from built-in types.

Recommended	Avoid
Preserve the traditional meaning of the operator.	Assign unrelated meanings to familiar operators.
Reuse existing operators to avoid duplicate logic.	Duplicate comparison or arithmetic code.
Return references where chaining is expected.	Return temporary objects unnecessarily.
Prefer member functions only when the left operand is the current object.	Force every operator to be a member.

At this point we have examined the most commonly overloaded operators used in everyday C++ programming. The remaining special member functions and advanced language features build upon the ideas introduced throughout this chapter.

COMMON PITFALLS WHEN OVERLOADING OPERATORS

Even when an overloaded operator compiles successfully, it may still produce confusing or unexpected behavior. Good operator design should make code easier to understand, not harder.

Pitfall	Recommendation
Changing the expected meaning of an operator.	Keep behavior consistent with built-in types.
Returning values instead of references where chaining is expected.	Return references for assignment, subscript and compound assignment operators.
Duplicating logic.	Implement one operator in terms of another.
Ignoring const correctness.	Declare read-only operators as const.

SELECTING THE RIGHT OPERATOR

Choose an operator only when its meaning is immediately obvious.

```
// Clear
Playlist merged = favorites + classics;

// Less clear
Playlist merged = favorites ^ classics;
```

The first expression communicates its intent immediately. The second requires readers to learn a new meaning for an existing operator.

SUMMARY

In this unit we learned how overloaded operators make user-defined classes integrate naturally with the C++ language. We implemented comparison, stream, arithmetic, subscript, function-call, unary, increment, decrement and conversion operators, discussed member versus non-member implementations, used friend functions where appropriate, reused existing operators to avoid duplicated logic, and followed the conventions established by built-in types.

REVIEW QUESTIONS

1. What is operator overloading?
2. When should an operator be a member function?
3. When is a `friend` operator appropriate?
4. Why should `operator!=` reuse `operator==`?
5. Why does `operator<<` return `ostream&`?
6. Why should `operator[ ]` return a reference?
7. What is the difference between prefix and postfix increment?
8. What is the purpose of explicit conversion operators?
9. Name three operators that cannot be overloaded.
10. What characteristics make an overloaded operator well designed?

LOOKING AHEAD

The next unit introduces inheritance. We will learn how to derive new classes from existing ones, reuse implementations, extend behavior, and model relationships between related objects while preserving good object-oriented design.